

实验报告：【Lab01】Storage HTTP

姓名 张昕跃

学号 2023202300

仓库 <https://github.com/pixels-courses/lab01-storage-http-imio21777>

实验核心要求

- 报告篇幅限制在 10 页以内。
- 报告需明确标注 AI 辅助完成的部分（如代码生成、文本润色）。
- 必须详细给出 AI 使用方式、技巧及心得体会，作为报告的一部分。
- 仓库请设置为私有，并在 Collaborators 添加助教 AntiO2, gengdy1545, Rolland1944, whzruc。

1 实验概述

实验背景是在不同进程之间，PixelsDB 的 Shuffle 框架依赖 mini-storage-http 在不同进程间转运列块。而 IO 文件夹下的 InputStream 和 OutputStream 的性能，直接影响后续 Shuffle 等场景的整体吞吐和资源占用。

原始实现虽然用了 Netty 和 AsyncHttpClient，但核心读写流程仍是串行、且存在大量无意义的忙等和数组拷贝，导致单机压测时 CPU 占用高、GC 压力大、吞吐也被往返延迟锁死。

本实验目的是依据 docs/lab01.md 提出的性能、资源和代码质量要求，在不改动公共接口的前提下重写输入流与输出流实现，把真正的异步管线化引入到 HTTP 传输路径中。

我的思路是在保持现有接口不变的前提下，重构读写：写端异步并行发送、读端用事件驱动。同时我用缓冲区作为一个新的数据结构，避免额外拷贝的出现和多次分配内存创建临时数组。语法上，加强错误处理和资源回收。具体来说，读端通过 Netty 事件驱动线程把每个 ByteBuf 直接投递到阻塞队列，消费端只需阻塞式地读取数据而不再轮询；写端利用复用缓冲块、有限在飞请求、失败重试和最终关闭握手把串行发送改造成真正的 pipeline。

2 实验内容

原先实现的 HttpOutputStream 只维护一个字节数组缓冲，调用线程必须同步等待每一次 HTTP 请求完成；而 HttpInputStream 则依赖固定间隔的休眠轮询去探测 BlockingQueue，即使 Netty 已写入数据也无法及时消费，连接提前关闭还会报错。同时还缺少统一的异常传播与关闭协议，一旦网络失败或对端断链，调用方只能无限等待，造成资源泄露。因此我主要做了三个改动：1. 减少拷贝；2. 真正异步；3. 完整的关闭与异常语义。

数据结构与算法设计

- 写端新增 BufferChunk，用于封装一段可复用的字节数组。配合 ConcurrentLinkedQueue 组成缓冲池，避免像原先版本那样对每个待发块都调用 Arrays.copyOfRange 频繁创建临时数组。数据写入后直接排队等待发送，无需额外拷贝。

- 读端新增 `ContentChunk`，其内部有 Netty 提供的 `ByteBuf`，也可被标记为 EOS (end of stream) 哨兵。利用 `AtomicBoolean remoteClosed`，消费者线程能够区分“暂时无数据”与“远端已关闭”两种状态。阻塞式 `poll` 替代原先的睡眠轮询，保证数据即到即取且不会重复释放。
- 发送由信号量 `inFlightSemaphore` 完成。`maxInflight` 个请求同时在途；失败重试则交给 `ScheduledExecutorService`，允许指数式地重新调度 `sendChunkAsync`。具体来说，用阻塞队列 + 信号量：写端通过 `LinkedBlockingQueue` 管理待发的数据块，再由单独的发送线程使用信号量限制在飞请求数量，实现真正的异步管线。
- 在异常处理和失败重试上：当网络请求失败时，使用 `ScheduledExecutorService` 以固定间隔重试，并在超过最大次数后上报错误。写端在所有数据发送完毕后，再同步等待 `Inflight` 结束并发送 `Connection: close`；读端在连接关闭、异常或 Netty 被动断开时都会接收到结束标记，保证资源回收完整。

OutputStream 代码逻辑

- `write(byte[],int,int)` 与 `write(int)` 都先执行 `checkFailure()` 和 `assertOpen()` 检查错误，然后在 `bufferLock` 保护下把数据写入 `activeChunk`。当 `remaining()==0` 时立即把当前块交给 `enqueueChunk`，并从缓冲池 `acquireChunk()` 获取新的写入对象。
- `flush()` 负责把尚未写满的 `activeChunk` 换出，以便后台线程立刻感知到部分数据；`close()` 先调用 `flush()`，再把剩余块入队并投递 `POISON_PILL`，同时等待 `terminationLatch` 确认发送线程完全退出。
- 独立的 `runSender()` 线程阻塞在 `contentQueue.take()`，消费到普通块时调用 `sendChunkAsync`；遇到 `poison` 则依次执行 `drainInflight()`（等待所有 `inflight` 释放）、`shutdownRetryExecutor()`（停止重试）以及 `sendCloseSignal()`。
- `sendChunkAsync` 通过 `inFlightSemaphore.acquire()` 控制并发度，再利用 `AsyncHttpClient` 构造 POST 请求。返回的 `ListenableFuture` 绑定到 `handleSendCompletion`，该回调根据 HTTP 状态或异常决定是否 `recycleChunk`、`scheduleRetry` 或 `recordFailure`，最后释放信号量。

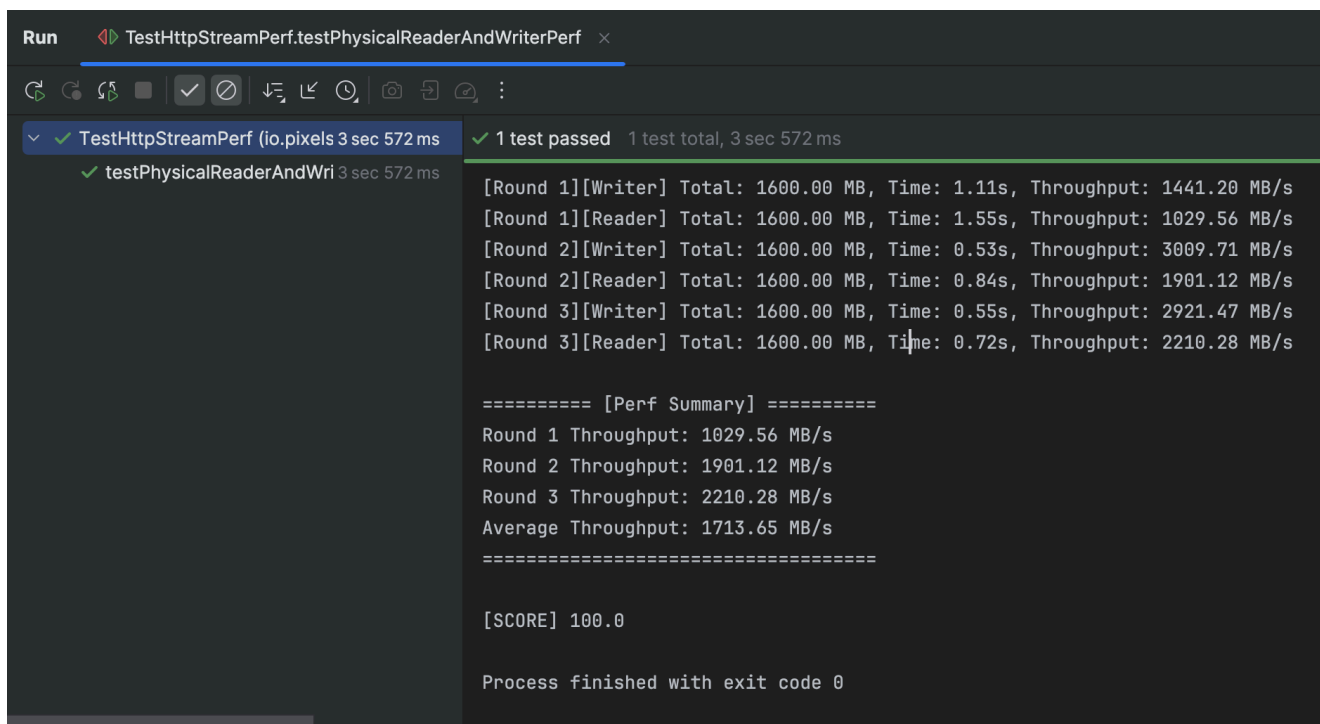
InputStream 代码逻辑

- `getReadableBuffer()` 会优先复用 `currentBuffer`；若为空或不可读，则调用 `pollNextBuffer()` 从 `BlockingQueue<ContentChunk>` 中阻塞式获取下一个数据块。
- `pollNextBuffer()` 在 `pollTimeoutMillis` 内等待队列元素。若拿到 `ContentChunk.end()`，仅把 `remoteClosed` 置位，并继续检查队列，确保所有缓存数据被消费后才真正返回 `null`。
- `read()` 与 `read(byte[],int,int)` 都基于上述缓冲：读完某个 `ByteBuf` 后调用 `releaseIfConsumed` 防止 Netty 引用泄露；若队列暂时为空但 `remoteClosed=false`，则继续阻塞等待。

- Netty 层的 `StreamHttpServerHandler.channelRead0()` 仅处理 POST 且 Content-Type 为 `application/x-protobuf` 的请求，读取到数据后调用 `content.retain()` 传 data；当对端请求关闭或管道异常时，`notifyEndOfStream()` 会入队 EOS，并通过 `onRemoteClose` 把关闭信号同步给消费者线程。

优化动机与效果这部分内容其实在前面思路和概述已经老生常谈地讲过了。

- 减少内存拷贝：复用 `BufferChunk` 避免频繁复制和 GC，读端也仅在必要时从 `ByteBuf` 拷贝到调用者数组。可以显著降低堆分配和 `Arrays.copyOfRange` 次数。
- 真正异步：写端通过后台发送线程 + in-flight 控制，实现调用线程、HTTP 请求、Close 握手三者；读端通过事件驱动 + 阻塞等待，压力测试下可以把单线程吞吐提升至受限于网络带宽而非 RTT。
- 可靠关闭与异常传播：统一的 `recordFailure/checkFailure` 让任意线程只需感知第一次失败即可，`drainPendingChunks` 和 `releaseIfConsumed` 则确保所有 Netty 引用都在关闭路径上被释放。



```
Run TestHttpStreamPerf.testPhysicalReaderAndWriterPerf x
✓ 1 test passed 1 test total, 3 sec 572 ms
✓ TestHttpStreamPerf (io.pixels 3 sec 572 ms)
  ✓ testPhysicalReaderAndWri 3 sec 572 ms

[Round 1][Writer] Total: 1600.00 MB, Time: 1.11s, Throughput: 1441.20 MB/s
[Round 1][Reader] Total: 1600.00 MB, Time: 1.55s, Throughput: 1029.56 MB/s
[Round 2][Writer] Total: 1600.00 MB, Time: 0.53s, Throughput: 3009.71 MB/s
[Round 2][Reader] Total: 1600.00 MB, Time: 0.84s, Throughput: 1901.12 MB/s
[Round 3][Writer] Total: 1600.00 MB, Time: 0.55s, Throughput: 2921.47 MB/s
[Round 3][Reader] Total: 1600.00 MB, Time: 0.72s, Throughput: 2210.28 MB/s

===== [Perf Summary] =====
Round 1 Throughput: 1029.56 MB/s
Round 2 Throughput: 1901.12 MB/s
Round 3 Throughput: 2210.28 MB/s
Average Throughput: 1713.65 MB/s
=====

[SCORE] 100.0

Process finished with exit code 0
```

图 1: mini-storage-http Perf Test1

```

Run TestHttpStreamPerf x
[Icons]
v TestHttpStreamPerf (io.pixelsdb.pixels.storage.http) 3 sec 562 ms t total, 3 sec 562 ms
  v testPhysicalReaderAndWri 3 sec 562 ms
    [Round 1][Writer] Total: 1600.00 MB, Time: 1.12s, Throughput: 1428.19 MB/s
    [Round 1][Reader] Total: 1600.00 MB, Time: 1.52s, Throughput: 1052.52 MB/s
    [Round 2][Writer] Total: 1600.00 MB, Time: 0.54s, Throughput: 2953.39 MB/s
    [Round 2][Reader] Total: 1600.00 MB, Time: 0.81s, Throughput: 1966.13 MB/s
    [Round 3][Writer] Total: 1600.00 MB, Time: 0.62s, Throughput: 2588.26 MB/s
    [Round 3][Reader] Total: 1600.00 MB, Time: 0.73s, Throughput: 2197.16 MB/s

    ===== [Perf Summary] =====
    Round 1 Throughput: 1052.52 MB/s
    Round 2 Throughput: 1966.13 MB/s
    Round 3 Throughput: 2197.16 MB/s
    Average Throughput: 1738.60 MB/s
    =====

    [SCORE] 100.0

    Process finished with exit code 0
  
```

图 2: mini-storage-http Perf Test2

```

Run TestHttpStreamPerf x
[Icons]
v TestHttpStreamPerf (io.pixels 3 sec 492 ms) v 1 test passed 1 test total, 3 sec 492 ms
  v testPhysicalReaderAndWri 3 sec 492 ms
    [Round 1][Writer] Total: 1600.00 MB, Time: 1.01s, Throughput: 1577.84 MB/s
    [Round 1][Reader] Total: 1600.00 MB, Time: 1.42s, Throughput: 1127.75 MB/s
    [Round 2][Writer] Total: 1600.00 MB, Time: 0.50s, Throughput: 3210.36 MB/s
    [Round 2][Reader] Total: 1600.00 MB, Time: 0.78s, Throughput: 2048.52 MB/s
    [Round 3][Writer] Total: 1600.00 MB, Time: 0.64s, Throughput: 2498.74 MB/s
    [Round 3][Reader] Total: 1600.00 MB, Time: 0.75s, Throughput: 2145.71 MB/s

    ===== [Perf Summary] =====
    Round 1 Throughput: 1127.75 MB/s
    Round 2 Throughput: 2048.52 MB/s
    Round 3 Throughput: 2145.71 MB/s
    Average Throughput: 1774.00 MB/s
    =====

    [SCORE] 100.0

    Process finished with exit code 0
  
```

图 3: mini-storage-http Perf Test3

3 实验总结

在改写过程中遇到的首要难题是“引入真正的并发发送”。最初尝试直接在调用线程里并发提交请求，但很快发现异常传播混乱、关闭顺序有问题（正确性不通过）。最终通过增加后台 `runSender` 线程和 `POISON_PILL` 哨兵，管理变量：调用方只需负责写数据和触发 `close`，其他细节全部交给发送线程收尾。

除此之外，还有改造忙等问题，阻塞式 `poll` 也用“空队列 + 远端已关闭”解决了忙等性能。

另一个问题是 Netty 侧的资源释放。老版本把 `ByteBuf` 直接塞入 `BlockingQueue`，消费者在异常路径下可能漏掉 `release()`。为此我明确引入了 `ContentChunk.end()` 哨兵与 `releaseIfConsumed` 等 helper funcs，保证无论正常读完还是异常终止都能成对地减少引用。我还使用 `AtomicBoolean` 标记和 `CountDownLatch` 进行同步，保证即使出现异常也不会造成死锁或资源泄露。

如果后续还要优化，可以考虑：

- 1) 在 `AsyncHttpClient` 层面共用连接池或开启 `HTTP/2`，以减少握手开销；
- 2) 根据写入速率动态调节 `maxInflight`，避免在慢网络上积压过多内存；
- 3) 为读取侧加入背压信号，把队列长度与写端 `inflight` 关联起来。

总体而言，本实验让我更加熟悉了基于 Netty 的流式传输模式，也体会到异步编程中资源管理与错误传播的重要性。最终的实现实现了真正的异步管线化、减少了内存拷贝，并改善了关闭与异常处理逻辑，为后续 `Shuffle` 场景提供了更高效的传输基础。本次实验交付了一个更贴近生产场景的流式 `HTTP` 通道：写端实现了真正的异步管线和失败治理，读端达成事件驱动的阻塞式消费，关闭路径和异常传播也变得可控。

4 AI 使用

AI 主要辅助完成了代码生成的过程和部分报告的撰写。我和 AI 提出了对应的代码需求，包括我的三个目标：1. 减少创建临时变量拷贝；2. 真正异步实现；3. 完整的关闭与异常语义。

在实现代码的过程中，AI 其实实现是非常冗余的，且过于精细化。主要体现在异常检查、资源回收这个部分。例如对于每个异常的检查，AI 会写的非常细，并且会出现多余的代码（但是不影响正确性），同时会拆分出很多函数，有点影响代码的易读性（太长了）。

在完成数据结构设计上，AI 倒是有很不错的表现，我告诉了 AI 使用缓冲区和阻塞队列改进目前的串行进程，它能够把这两个数据结构写的没有错 + 高性能。

真正的异步实现实际上在数据结构写完之后，对 AI 来说就只是加几句调用的事情，这个不是很难的地方。

最后是辅助报告生成。我先让 AI 生成了一个比较详尽的对代码的解释和文内注释，然后自己对着进行了一些重构。在实验概述上，和具体实验内容概述，总结都是我完成的。对于 AI 来说，我让它把文内关键的函数实现提取和每个函数逻辑讲明白，主要表现在实验内容中的设计和代码逻辑部分，然后在他的基础上做了一些正确性和实现的修改。

使用 AI 可以保证一个不会出错的 `Latex` 语法，也给我写报告带来了比较大的方便。

A 附录 1: HttpInputStream

```
1
2  /*
3   * Copyright 2024 PixelsDB.
4   *
5   * This file is part of Pixels.
6   *
7   * Pixels is free software: you can redistribute it and/or modify
8   * it under the terms of the Affero GNU General Public License as
9   * published by the Free Software Foundation, either version 3 of
10  * the License, or (at your option) any later version.
11  *
12  * Pixels is distributed in the hope that it will be useful,
13  * but WITHOUT ANY WARRANTY; without even the implied warranty of
14  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
15  * Affero GNU General Public License for more details.
16  *
17  * You should have received a copy of the Affero GNU General Public
18  * License along with Pixels. If not, see
19  * <https://www.gnu.org/licenses/>.
20  */
21 package io.pixelsdb.pixels.storage.http.io;
22
23 import io.netty.buffer.ByteBuf;
24 import io.netty.channel.ChannelHandlerContext;
25 import io.netty.handler.codec.http.*;
26 import io.pixelsdb.pixels.common.utils.Constants;
27 import org.apache.logging.log4j.LogManager;
28 import org.apache.logging.log4j.Logger;
29
30 import javax.net.ssl.SSLException;
31 import java.io.IOException;
32 import java.io.InputStream;
33 import java.security.cert.CertificateException;
34 import java.util.concurrent.*;
35 import java.util.concurrent.atomic.AtomicBoolean;
36
37 public class HttpInputStream extends InputStream
38 {
39     private static final Logger logger = LogManager.getLogger(HttpInputStream.class);
40 }
```

```
41  /**
42   * indicates whether the http is still open / valid
43   */
44  private boolean open;
45
46  /**
47   * The schema of http.
48   * Default value is http.
49   */
50  private final String schema = "http";
51
52  /**
53   * The host of http.
54   */
55  private String host;
56
57  /**
58   * The port of http.
59   */
60  private int port;
61
62  /**
63   * The uri of http.
64   */
65  private String uri;
66
67  /**
68   * The temporary buffer used for storing the chunks.
69   */
70  private final BlockingQueue<ContentChunk> contentQueue;
71
72  /**
73   * The maximum tries to get data.
74   */
75  private final int MAX_TRIES = Constants.MAX_STREAM_RETRY_COUNT;
76
77  /**
78   * The milliseconds to sleep.
79   */
80  private final long DELAY_MS = Constants.STREAM_DELAY_MS;
81
82  /**
83   * The http server for receiving input http.
```

```

84     */
85     private final HttpServer httpServer;
86
87     /**
88      * The thread to run http server.
89      */
90     private final ExecutorService executorService;
91
92     /**
93      * The future of http server.
94      */
95     private final CompletableFuture<Void> httpServerFuture;
96
97     // transfer between main thread and Netty server
98     private final AtomicBoolean remoteClosed;
99     // timeout for polling data from the queue
100    private final long pollTimeoutMillis;
101    // buffer for storing the current Netty chunk
102    private ByteBuf currentBuffer;
103
104    // constructor
105    public HttpInputStream(String host, int port) throws CertificateException,
        ↳ SSLEException {
106        this.open = true;
107        this.contentQueue = new LinkedBlockingQueue<>();
108        this.host = host;
109        this.port = port;
110        this.uri = this.schema + "://" + host + ":" + port;
111        this.remoteClosed = new AtomicBoolean(false);
112        this.pollTimeoutMillis = Math.max(1L, Constants.STREAM_DELAY_MS);
113        this.httpServer = new HttpServer(new
            ↳ StreamHttpServerHandler(this.contentQueue, this::onRemoteClosed));
114        this.executorService =
            ↳ Executors.newSingleThreadExecutor(namedThreadFactory("httpstream-server"));
115        this.httpServerFuture = CompletableFuture.runAsync(() -> {
116            try
117            {
118                this.httpServer.serve(this.port);
119            } catch (InterruptedException e)
120            {
121                Thread.currentThread().interrupt();
122                logger.warn("http server interrupted", e);
123            }

```

```

124         }, this.executorService);
125     }
126
127     @Override
128     // 先调用 assertOpen() 确保流未关闭, 再用 getReadableBuffer() 拿到可读的 ByteBuf; 若
129     //   ↳ 为 null 则返回 -1 表示 EOF。
130     // 读完后调用 releaseIfConsumed(buffer) 在缓冲耗尽时释放 Netty 引用。
131     public int read() throws IOException
132     {
133         assertOpen();
134         // get readable buffer
135         ByteBuf buffer = getReadableBuffer();
136         if (buffer == null)
137         {
138             return -1;
139         }
140         int value = buffer.readUnsignedByte();
141         // release buffer
142         releaseIfConsumed(buffer);
143         return value;
144     }
145
146     @Override
147     public int read(byte[] b) throws IOException
148     {
149         return read(b, 0, b.length);
150     }
151
152     /**
153      * Attempt to read data with a maximum length of len into the position off of buf.
154      * @param buf
155      * @param off
156      * @param len
157      * @return Actual number of bytes read
158      * @throws IOException
159      */
160     @Override
161     // 循环填充用户缓冲区: readBytes 记录累计读取量; toRead 是本次最多可读的字节数 (剩余需
162     //   ↳ 求与缓冲区可读长度中的较小者)。
163     // 发生运行时异常时立即释放当前 buffer 后抛出。全部读取完成或遇到 EOF 时返回相应字节
164     //   ↳ 数。
165     public int read(byte[] buf, int off, int len) throws IOException
166     {

```

```
164         assertOpen();
165
166         int readBytes = 0;
167         while (readBytes < len)
168         {
169             ByteBuf buffer = getReadableBuffer();
170             if (buffer == null)
171             {
172                 return readBytes > 0 ? readBytes : -1;
173             }
174             int toRead = Math.min(len - readBytes, buffer.readableBytes());
175             try
176             {
177                 buffer.readBytes(buf, off + readBytes, toRead);
178                 readBytes += toRead;
179             }
180             catch (RuntimeException e)
181             {
182                 releaseIfConsumed(buffer);
183                 throw e;
184             }
185             releaseIfConsumed(buffer);
186         }
187
188         return readBytes;
189     }
190
191     @Override
192     public void close() throws IOException
193     {
194         // 标记 open=false, 通过 releaseCurrentBuffer() 释放正在使用的 ByteBuf,
195         // ↪ drainPendingChunks() 清空队列里还未消费的内容块。
196         if (this.open)
197         {
198             this.open = false;
199             releaseCurrentBuffer();
200             drainPendingChunks();
201             this.httpServer.close();
202             // shutdown 处理
203             this.executorService.shutdownNow();
204             try
205             {
206                 this.httpServerFuture.get(5, TimeUnit.SECONDS);
207             }
208             catch (InterruptedException e)
209             {
210                 // 忽略
211             }
212         }
213     }
214 }
```

```
206         }
207         catch (InterruptedException e)
208         {
209             Thread.currentThread().interrupt();
210         }
211         catch (TimeoutException | java.util.concurrent.ExecutionException e)
212         {
213             logger.warn("http server shutdown wait failed", e);
214         }
215     }
216 }
217
218 // helper funcs
219 private boolean emptyData() throws IOException
220 {
221     int tries = 0;
222     while (tries < this.MAX_TRIES && this.contentQueue.isEmpty() &&
223           ↪ !this.httpServerFuture.isDone())
224     {
225         try
226         {
227             tries++;
228             Thread.sleep(this.DELAY_MS);
229         } catch (InterruptedException e)
230         {
231             throw new IOException(e);
232         }
233     }
234     if (tries == this.MAX_TRIES)
235     {
236         logger.error("retry count {}, httpServerFuture {}, " +
237           ↪ "exception cause: HttpInputStream failed to receive data", tries,
238           ↪ this.httpServerFuture.isDone());
239     }
240     return this.contentQueue.isEmpty();
241 }
242
243 private void assertOpen()
244 {
245     if (!this.open)
246     {
247         throw new IllegalStateException("Closed");
248     }
249 }
```

```
247     }
248 }
249
250 private void drainPendingChunks()
251 {
252     ContentChunk chunk;
253     while ((chunk = contentQueue.poll()) != null)
254     {
255         if (!chunk.isEndOfStream())
256         {
257             chunk.buffer().release();
258         }
259     }
260 }
261
262 private ByteBuf getReadableBuffer() throws IOException
263 {
264     if (currentBuffer != null && currentBuffer.isReadable())
265     {
266         return currentBuffer;
267     }
268     releaseCurrentBuffer();
269     currentBuffer = pollNextBuffer();
270     return currentBuffer;
271 }
272
273
274 private ByteBuf pollNextBuffer() throws IOException
275 {
276     while (open)
277     {
278         ContentChunk chunk;
279         try
280         {
281             chunk = contentQueue.poll(pollTimeoutMillis, TimeUnit.MILLISECONDS);
282         }
283         catch (InterruptedException e)
284         {
285             Thread.currentThread().interrupt();
286             throw new IOException("interrupted while waiting for http content",
287                                     ↪ e);
288         }
289     }
290     if (chunk == null)
```

```
289         {
290             if (remoteClosed.get() && contentQueue.isEmpty())
291             {
292                 return null;
293             }
294             continue;
295         }
296         if (chunk.isEndOfStream())
297         {
298             remoteClosed.set(true);
299             continue;
300         }
301         return chunk.buffer();
302     }
303     return null;
304 }
305
306 private void releaseIfConsumed(ByteBuf buffer)
307 {
308     if (buffer != null && !buffer.isReadable())
309     {
310         buffer.release();
311         if (buffer == currentBuffer)
312         {
313             currentBuffer = null;
314         }
315     }
316 }
317
318 private void releaseCurrentBuffer()
319 {
320     if (currentBuffer != null)
321     {
322         currentBuffer.release();
323         currentBuffer = null;
324     }
325 }
326
327 private void onRemoteClosed()
328 {
329     remoteClosed.set(true);
330 }
331
```

```
332 private static ThreadFactory namedThreadFactory(String name)
333 {
334     return runnable -> {
335         Thread thread = new Thread(runnable, name);
336         thread.setDaemon(true);
337         return thread;
338     };
339 }
340
341 // helper data structure
342 private static final class ContentChunk
343 {
344     private final ByteBuf buffer;
345     private final boolean endOfStream;
346
347     private ContentChunk(ByteBuf buffer, boolean endOfStream)
348     {
349         this.buffer = buffer;
350         this.endOfStream = endOfStream;
351     }
352
353     static ContentChunk data(ByteBuf buffer)
354     {
355         return new ContentChunk(buffer, false);
356     }
357
358     static ContentChunk end()
359     {
360         return new ContentChunk(null, true);
361     }
362
363     boolean isEndOfStream()
364     {
365         return endOfStream;
366     }
367
368     ByteBuf buffer()
369     {
370         return buffer;
371     }
372 }
373
374
```

```

375 public static class StreamHttpServerHandler extends HttpServerHandler
376 {
377     private static final Logger logger =
378         ↪ LogManager.getLogger(StreamHttpServerHandler.class);
379     private final BlockingQueue<ContentChunk> contenQueue; // 存放从 HTTP 请求中提
380         ↪ 取出的数据块
381     private final Runnable onRemoteClose; // HTTP 连接被动关闭时的通知回调
382     private final AtomicBoolean eosSent = new AtomicBoolean(false); // 保证仅发送一
383         ↪ 次流结束标记
384
385     public StreamHttpServerHandler(BlockingQueue<ContentChunk> contenQueue,
386         ↪ Runnable onRemoteClose)
387     {
388         this.contenQueue = contenQueue;
389         this.onRemoteClose = onRemoteClose;
390     }
391
392     @Override
393     public void channelRead0(ChannelHandlerContext ctx, HttpObject msg)
394     {
395         // 只处理 FullHttpRequest, 其余对象直接忽略
396         if (!(msg instanceof FullHttpRequest))
397         {
398             return;
399         }
400         FullHttpRequest req = (FullHttpRequest) msg;
401         // 仅允许 POST 写入数据, 其余方法直接返回 405
402         if (req.method() != HttpMethod.POST)
403         {
404             sendResponse(ctx, req, HttpResponseStatus.METHOD_NOT_ALLOWED, false);
405             return;
406         }
407
408         // 限定 Content-Type, 避免接收错误格式的数据
409         CharSequence contentType =
410             ↪ req.headers().get(HttpHeaderNames.CONTENT_TYPE);
411         if (contentType == null ||
412             ↪ !"application/x-protobuf".equals(contentType))
413         {
414             sendResponse(ctx, req, HttpResponseStatus.UNSUPPORTED_MEDIA_TYPE,
415                 ↪ false);
416             return;
417         }
418     }

```

```
411     ByteBuf content = req.content();
412     if (content.isReadable())
413     {
414         // retain 递增引用计数，数据交由消费线程释放
415         content.retain();
416         this.contentQueue.add(ContentChunk.data(content));
417     }
418
419     boolean close = isConnectionClose(req.headers());
420     sendResponse(ctx, req, HttpResponseStatus.OK, close);
421 }
422
423 private boolean isConnectionClose(HttpHeaders headers)
424 {
425     // 根据 Connection 头判断客户端是否要求关闭连接
426     CharSequence connection = headers.get(HttpHeaderNames.CONNECTION);
427     return connection != null &&
428         ↪ HttpHeaders.Values.CLOSE.contentEqualsIgnoreCase(connection);
429 }
430
431 private void sendResponse(ChannelHandlerContext ctx, FullHttpRequest req,
432     ↪ HttpResponseStatus status, boolean close)
433 {
434     // 回写简单的文本应答，客户端只需关注状态码
435     FullHttpResponse response = new
436         ↪ DefaultFullHttpResponse(req.protocolVersion(), status);
437     response.headers()
438         .set(HttpHeaderNames.CONTENT_TYPE, "text/plain")
439         .set(HttpHeaderNames.CONTENT_LENGTH,
440             ↪ response.content().readableBytes());
441
442     if (close)
443     {
444         // 如果客户端希望关闭连接，则发送响应后立刻标记流结束
445         response.headers().set(HttpHeaderNames.CONNECTION,
446             ↪ HttpHeaders.Values.CLOSE);
447         response.setStatus(status);
448         ctx.writeAndFlush(response);
449         notifyEndOfStream();
450         this.serverCloser.run();
451     } else
452     {
453         // 否则保持长连接等待后续请求
454     }
455 }
```

```

449         response.headers().set(HttpHeaderNames.CONNECTION,
           ↳ HttpHeaders.Values.KEEP_ALIVE);
450         response.setStatus(status);
451         ctx.writeAndFlush(response);
452     }
453 }
454
455 @Override
456 public void channelInactive(ChannelHandlerContext ctx) throws Exception
457 {
458     super.channelInactive(ctx);
459     // 连接断开时可能没有 POST 请求，因此主动告诉消费端流结束
460     notifyEndOfStream();
461 }
462
463 @Override
464 public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause)
465 {
466     super.exceptionCaught(ctx, cause);
467     // 出现异常同样触发 EOS，避免读端永久阻塞
468     notifyEndOfStream();
469 }
470
471 private void notifyEndOfStream()
472 {
473     // 仅第一次触发时入队 EOS，并执行关闭回调
474     if (this.eosSent.compareAndSet(false, true))
475     {
476         this.contentQueue.add(ContentChunk.end());
477         if (this.onRemoteClose != null)
478         {
479             this.onRemoteClose.run();
480         }
481     }
482 }
483 }
484 }
485

```

B 附录 2: HttpOutputputStream

```
1  /*
2   * Copyright 2024 PixelsDB.
3   *
4   * This file is part of Pixels.
5   *
6   * Pixels is free software: you can redistribute it and/or modify
7   * it under the terms of the Affero GNU General Public License as
8   * published by the Free Software Foundation, either version 3 of
9   * the License, or (at your option) any later version.
10  *
11  * Pixels is distributed in the hope that it will be useful,
12  * but WITHOUT ANY WARRANTY; without even the implied warranty of
13  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
14  * Affero GNU General Public License for more details.
15  *
16  * You should have received a copy of the Affero GNU General Public
17  * License along with Pixels. If not, see
18  * <https://www.gnu.org/licenses/>.
19  */
20 package io.pixelsdb.pixels.storage.http.io;
21
22 import io.netty.handler.codec.http.HttpHeaderNames;
23 import io.netty.handler.codec.http.HttpHeaderValues;
24 import io.pixelsdb.pixels.common.utils.Constants;
25 import org.apache.logging.log4j.LogManager;
26 import org.apache.logging.log4j.Logger;
27 import org.asynchttpclient.*;
28
29 import java.io.IOException;
30 import java.io.OutputStream;
31 import java.nio.ByteBuffer;
32 import java.util.Arrays;
33 import java.util.concurrent.*;
34 import java.util.concurrent.atomic.AtomicBoolean;
35 import java.util.concurrent.atomic.AtomicReference;
36
37 public class HttpOutputStream extends OutputStream
38 {
39     private static final Logger logger = LogManager.getLogger(HttpOutputStream.class);
40 }
```

```
41  /**
42   * indicates whether the http is still open / valid
43   */
44  private boolean open;
45
46  /**
47   * The schema of http.
48   * Default value is http.
49   */
50  private final String schema = "http";
51
52  /**
53   * The host of http.
54   */
55  private String host;
56
57  /**
58   * The port of http.
59   */
60  private int port;
61
62  /**
63   * The uri of http.
64   */
65  private String uri;
66
67  /**
68   * The maximum retry count.
69   */
70  private static final int MAX_RETRIES = Constants.MAX_STREAM_RETRY_COUNT;
71
72  /**
73   * The delay between two tries.
74   */
75  private static final long RETRY_DELAY_MS = Constants.STREAM_DELAY_MS;
76
77  // /**
78  //  * The temporary buffer used for storing the chunks.
79  //  */
80  // private final byte[] buffer;
81
82  // /**
83  //  * The position in the buffer.
```

```

84     // */
85     // private int bufferPosition;
86
87     /**
88      * The capacity of buffer.
89      */
90     private int bufferCapacity;
91
92     // /**
93     //  * The background thread to send requests.
94     //  */
95     // private final ExecutorService executorService;
96
97     /**
98      * The http client.
99      */
100    private final AsyncHttpClient httpClient;
101
102    /**
103     * The queue to put pending requests.
104     */
105    private final BlockingQueue<BufferChunk> contentQueue;
106
107    private final int maxInflight; // 允许同时存在的 HTTP 请求数
108    private final ConcurrentLinkedQueue<BufferChunk> bufferPool; // 可复用的块池，避免
    ↪ 频繁分配
109    private final ExecutorService senderExecutor; // 负责消费队列并发送 HTTP 的线程
110    private final ScheduledExecutorService retryExecutor; // 用于延迟重试的调度线程
111    private final Semaphore inFlightSemaphore; // 控制在途请求数量
112    private final CountDownLatch terminationLatch; // 用于等待发送线程结束
113    private final AtomicReference<Throwable> failure; // 记录第一起异常
114    private final AtomicBoolean closeInitiated; // 防止 close 重入
115    private final Object bufferLock = new Object(); // 保护 activeChunk 的锁
116    private BufferChunk activeChunk; // 当前正在填充的 BufferChunk
117    private static final BufferChunk POISON_PILL = BufferChunk.poisson();
118
119    // constructor
120    public HttpOutputStream(String host, int port, int bufferCapacity)
121    {
122        this.open = true;
123        this.host = host;
124        this.port = port;
125        this.uri = "http://" + host + ":" + port;

```

```

126     this.bufferCapacity = bufferCapacity;
127     this.maxInflight = Math.max(1,
    ↪ Integer.getInteger("pixels.httpstream.maxInflight", 4));
128     this.httpClient = Dsl.asyncHttpClient();
129     this.contentQueue = new LinkedBlockingQueue<>();
130     this.bufferPool = new ConcurrentLinkedQueue<>();
131     this.inFlightSemaphore = new Semaphore(this.maxInflight);
132     this.terminationLatch = new CountDownLatch(1);
133     this.failure = new AtomicReference<>();
134     this.closeInitiated = new AtomicBoolean(false);
135     this.senderExecutor =
    ↪ Executors.newSingleThreadExecutor(namedThreadFactory("httpstream-sender"));
136     this.retryExecutor =
    ↪ Executors.newSingleThreadScheduledExecutor(namedThreadFactory("httpstream-retry"));
137     this.activeChunk = acquireChunk();
138
139     // 从队列读取 BufferChunk 并通过 HTTP 发送
140     this.senderExecutor.execute(this::runSender);
141 }
142
143 /**
144  * Write an array to the S3 output http
145  *
146  * @param b
147  * @throws IOException
148  */
149 @Override
150 public void write(byte[] b) throws IOException
151 {
152     write(b, 0, b.length);
153 }
154
155 @Override
156 public void write(final byte[] buf, final int off, final int len) throws
    ↪ IOException
157 {
158     // handle errors
159     if (buf == null)
160     {
161         throw new NullPointerException("buffer");
162     }
163     if (off < 0 || len < 0 || off + len > buf.length)
164     {

```

```
165         throw new IndexOutOfBoundsException("Invalid offset/length");
166     }
167     if (len == 0)
168     {
169         return;
170     }
171     checkFailure();
172     assertOpen();
173
174     int offsetInBuf = off, remainToRead = len;
175     while (remainToRead > 0)
176     {
177         BufferChunk readyBeforeCopy = null; // 在写入前已满，需要入队
178         BufferChunk readyAfterCopy = null; // 写完后刚好被填满的块
179         int toCopy;
180         synchronized (bufferLock)
181         {
182             ensureActiveChunk();
183             if (activeChunk.remaining() == 0)
184             {
185                 readyBeforeCopy = activeChunk;
186                 activeChunk = acquireChunk();
187             }
188             toCopy = Math.min(activeChunk.remaining(), remainToRead);
189             System.arraycopy(buf, offsetInBuf, activeChunk.data,
190                 ↪ activeChunk.length, toCopy);
191             activeChunk.length += toCopy;
192             if (activeChunk.remaining() == 0)
193             {
194                 readyAfterCopy = activeChunk;
195                 activeChunk = acquireChunk();
196             }
197             enqueueChunk(readyBeforeCopy);
198             enqueueChunk(readyAfterCopy);
199             offsetInBuf += toCopy;
200             remainToRead -= toCopy;
201         }
202     }
203
204     @Override
205     public void write(int b) throws IOException
206     {
```

```
207     checkFailure();
208     assertOpen();
209     BufferChunk readyBeforeCopy = null;
210     BufferChunk readyAfterCopy = null;
211     // 锁同步
212     synchronized (bufferLock)
213     {
214         ensureActiveChunk();
215         if (activeChunk.remaining() == 0)
216         {
217             readyBeforeCopy = activeChunk;
218             activeChunk = acquireChunk();
219         }
220         activeChunk.data[activeChunk.length++] = (byte) b;
221         if (activeChunk.remaining() == 0)
222         {
223             readyAfterCopy = activeChunk;
224             activeChunk = acquireChunk();
225         }
226     }
227     enqueueChunk(readyBeforeCopy);
228     enqueueChunk(readyAfterCopy);
229 }
230
231 @Override
232 public synchronized void flush() throws IOException
233 {
234     checkFailure();
235     BufferChunk ready = null;
236     synchronized (bufferLock)
237     {
238         if (!open)
239         {
240             return;
241         }
242         if (activeChunk != null && activeChunk.length > 0)
243         {
244             ready = activeChunk; // 将部分填充的块也提交发送
245             activeChunk = acquireChunk();
246         }
247     }
248     enqueueChunk(ready);
249 }
```

```
250
251 // protected void flushBufferAndRewind() throws IOException
252 // {
253 //     logger.debug("Sending {} bytes to http", this.bufferPosition);
254 //     byte[] content = Arrays.copyOfRange(this.buffer, 0, this.bufferPosition);
255 //     this.bufferPosition = 0;
256 //     this.contentQueue.add(content);
257 // }
258
259 @Override
260 public void close() throws IOException
261 {
262
263     if (!closeInitiated.compareAndSet(false, true))
264     {
265         // 已经有线程在关闭，等待其结束并抛出记录的异常
266         waitForTermination();
267         throwFailureIfAny();
268         return;
269     }
270
271     try
272     {
273         flush();
274     }
275     catch (IOException e)
276     {
277         // 关闭阶段也记录异常，最终在 throwFailureIfAny 中抛出
278         recordFailure(e);
279     }
280
281     this.open = false;
282     BufferChunk leftover;
283     synchronized (bufferLock)
284     {
285         leftover = this.activeChunk;
286         this.activeChunk = null;
287     }
288     if (leftover != null)
289     {
290         if (leftover.length > 0)
291         {
292             // 还有未发送数据则入队
```

```
293         enqueueChunk(leftover);
294     }
295     else
296     {
297         recycleChunk(leftover);
298     }
299 }
300
301 // POISON_PILL 会让发送线程在 runSender 中退出
302 enqueueChunk(POISON_PILL);
303 waitForTermination();
304 retryExecutor.shutdownNow();
305 senderExecutor.shutdownNow();
306 throwFailureIfAny();
307 }
308
309 private void handleSendCompletion(BufferChunk chunk, ListenableFuture<Response>
    ↪ future, int attempt)
310 {
311     boolean rescheduled = false;
312     Throwable failureCause = null;
313     Response response = null;
314     try
315     {
316         response = future.get();
317     }
318     catch (InterruptedException e)
319     {
320         Thread.currentThread().interrupt();
321         failureCause = e;
322     }
323     catch (ExecutionException e)
324     {
325         failureCause = unwrap(e);
326     }
327     finally
328     {
329         if (failureCause == null && response != null)
330         {
331             int status = response.getStatusCode();
332             if (status < 200 || status >= 300)
333             {
334                 // HTTP 非 2xx 同样视为失败，交给重试或失败处理
```

```
335         failureCause = new IOException("Unexpected response status: " +
336             ↪ status);
337     }
338
339     if (failureCause == null)
340     {
341         recycleChunk(chunk);
342     }
343     else
344     {
345         if (shouldRetry(failureCause, attempt))
346         {
347             rescheduled = scheduleRetry(chunk, attempt + 1);
348         }
349         if (!rescheduled)
350         {
351             // 无法重试则记录失败，并释放当前块以避免泄漏
352             recordFailure(failureCause);
353             recycleChunk(chunk);
354         }
355     }
356     inFlightSemaphore.release();
357 }
358
359
360
361 // helper data structure
362 private static final class BufferChunk
363 {
364     private final byte[] data;
365     private int length;
366     private final boolean poison;
367
368     private BufferChunk(int capacity)
369     {
370         this.data = new byte[capacity];
371         this.length = 0;
372         this.poison = false;
373     }
374
375     private BufferChunk(boolean poison)
376     {
```

```
377         this.data = null;
378         this.length = 0;
379         this.poison = poison;
380     }
381
382     static BufferChunk data(int capacity)
383     {
384         return new BufferChunk(capacity);
385     }
386
387     static BufferChunk poison()
388     {
389         return new BufferChunk(true);
390     }
391
392     int remaining()
393     {
394         return this.poison ? 0 : (this.data.length - this.length);
395     }
396
397     ByteBuffer toByteBuffer()
398     {
399         return ByteBuffer.wrap(this.data, 0, this.length);
400     }
401
402     void reset()
403     {
404         if (!this.poison)
405         {
406             this.length = 0;
407         }
408     }
409
410     boolean isPoison()
411     {
412         return this.poison;
413     }
414 }
415
416
417
418 // helper functions
419 private void runSender()
```

```
420 {
421     try
422     {
423         while (true)
424         {
425             BufferChunk chunk = contentQueue.take();
426             if (chunk.isPoison())
427             {
428                 // 收到 poison 表示写端已结束，等待在途请求并通知对端关闭
429                 drainInflight();
430                 shutdownRetryExecutor();
431                 sendCloseSignal();
432                 break;
433             }
434             // 正常数据块异步发送
435             sendChunkAsync(chunk, 0);
436         }
437     }
438     catch (InterruptedException e)
439     {
440         Thread.currentThread().interrupt();
441         recordFailure(e);
442     }
443     finally
444     {
445         try
446         {
447             httpClient.close();
448         }
449         catch (IOException e)
450         {
451             logger.warn("Failed to close http client", e);
452         }
453         terminationLatch.countDown();
454     }
455 }
456
457 private void shutdownRetryExecutor()
458 {
459     retryExecutor.shutdown();
460     try
461     {
```

```
462         if (!retryExecutor.awaitTermination(RETRY_DELAY_MS * (MAX_RETRIES + 1L),
463             ↪ TimeUnit.MILLISECONDS))
464         {
465             logger.warn("Timed out waiting for retry tasks to finish");
466         }
467     catch (InterruptedException e)
468     {
469         Thread.currentThread().interrupt();
470     }
471 }
472
473 private void sendChunkAsync(BufferChunk chunk, int attempt)
474 {
475     if (!awaitPermit())
476     {
477         // 获取并发许可失败 (一般是线程被中断), 直接回收该块
478         recycleChunk(chunk);
479         return;
480     }
481
482     Request request = httpClient.preparePost(this.uri)
483         .setBody(chunk.toByteBuffer())
484         .addHeader(HttpHeaderNames.CONTENT_TYPE, "application/x-protobuf")
485         .addHeader(HttpHeaderNames.CONTENT_LENGTH,
486             ↪ String.valueOf(chunk.length))
487         .addHeader(HttpHeaderNames.CONNECTION,
488             ↪ HttpHeaders.Values.KEEP_ALIVE.toString())
489         .build();
490
491     // 监听发送结果, 必要时重试
492     ListenableFuture<Response> future = httpClient.executeRequest(request);
493     future.addListener(() -> handleSendCompletion(chunk, future, attempt),
494         ↪ Runnable::run);
495 }
496
497 private boolean scheduleRetry(BufferChunk chunk, int nextAttempt)
498 {
499     try
500     {
501         // 通过调度线程延迟再次调用 sendChunkAsync
502         retryExecutor.schedule(() -> sendChunkAsync(chunk, nextAttempt),
503             ↪ RETRY_DELAY_MS, TimeUnit.MILLISECONDS);
504     }
```

```
500         return true;
501     }
502     catch (RejectedExecutionException ex)
503     {
504         return false;
505     }
506 }
507
508 private boolean shouldRetry(Throwable throwable, int attempt)
509 {
510     if (attempt >= MAX_RETRIES)
511     {
512         return false;
513     }
514     Throwable cause = unwrap(throwable);
515     // 只对典型的网络 / IO 异常进行重试
516     return cause instanceof java.net.ConnectException || cause instanceof
        ↳ IOException;
517 }
518
519 private Throwable unwrap(Throwable throwable)
520 {
521     Throwable current = throwable;
522     while (current instanceof ExecutionException && current.getCause() != null)
523     {
524         current = current.getCause();
525     }
526     return current;
527 }
528
529 private boolean awaitPermit()
530 {
531     try
532     {
533         // 控制并发的核心，每个在途请求占用一个许可
534         inFlightSemaphore.acquire();
535         return true;
536     }
537     catch (InterruptedException e)
538     {
539         Thread.currentThread().interrupt();
540         recordFailure(e);
541         return false;
542     }
543 }
```

```
542     }
543 }
544
545 private void drainInflight()
546 {
547     try
548     {
549         // 通过一次性获取全部许可，等待所有 in-flight 请求完成
550         inFlightSemaphore.acquire(this.maxInflight);
551         inFlightSemaphore.release(this.maxInflight);
552     }
553     catch (InterruptedException e)
554     {
555         Thread.currentThread().interrupt();
556     }
557 }
558
559 private void sendCloseSignal()
560 {
561     for (int attempt = 0; attempt <= MAX_RETRIES; attempt++)
562     {
563         // 发送一个 Content-Length=0 且 Connection: close 的 POST，告知对端终止
564         Request request = httpClient.preparePost(this.uri)
565             .addHeader(HttpHeaderNames.CONTENT_TYPE, "application/x-protobuf")
566             .addHeader(HttpHeaderNames.CONTENT_LENGTH, "0")
567             .addHeader(HttpHeaderNames.CONNECTION, HttpHeaders.Values.CLOSE)
568             .build();
569         try
570         {
571             httpClient.executeRequest(request).get();
572             return;
573         }
574         catch (Exception e)
575         {
576             if (!shouldRetry(e, attempt))
577             {
578                 logger.warn("Failed to close http reader after {} attempts",
579                     ↪ attempt + 1, e);
580                 return;
581             }
582             sleepQuietly(RETRY_DELAY_MS);
583         }
584     }
585 }
```

```
584     }
585
586     private void ensureActiveChunk()
587     {
588         if (activeChunk == null)
589         {
590             // 没有可写块时从池中取一个
591             activeChunk = acquireChunk();
592         }
593     }
594
595     private BufferChunk acquireChunk()
596     {
597         BufferChunk chunk = bufferPool.poll();
598         if (chunk == null)
599         {
600             chunk = BufferChunk.data(bufferCapacity);
601         }
602         chunk.reset();
603         return chunk;
604     }
605
606     private void recycleChunk(BufferChunk chunk)
607     {
608         if (chunk == null || chunk.isPoison())
609         {
610             return;
611         }
612         chunk.reset();
613         bufferPool.offer(chunk); // 回收到池里，减少 GC
614     }
615
616     private void enqueueChunk(BufferChunk chunk)
617     {
618         if (chunk != null)
619         {
620             // 生产者只需入队，runSender 会负责后续发送
621             contentQueue.add(chunk);
622         }
623     }
624
625     private void assertOpen() throws IOException
626     {
```

```
627         if (!open)
628         {
629             throw new IOException("Stream closed");
630         }
631     }
632
633     private void checkFailure() throws IOException
634     {
635         Throwable throwable = failure.get();
636         if (throwable != null)
637         {
638             throw wrapAsIOException(throwable);
639         }
640     }
641
642     private void recordFailure(Throwable throwable)
643     {
644         if (throwable == null)
645         {
646             return;
647         }
648         if (failure.compareAndSet(null, unwrap(throwable)))
649         {
650             logger.error("HTTP output stream failure: {}", throwable.getMessage(),
651                 ↪ throwable);
652         }
653     }
654
655     private void throwFailureIfAny() throws IOException
656     {
657         Throwable throwable = failure.get();
658         if (throwable != null)
659         {
660             throw wrapAsIOException(throwable);
661         }
662     }
663
664     private IOException wrapAsIOException(Throwable throwable)
665     {
666         if (throwable instanceof IOException)
667         {
668             return (IOException) throwable;
```

```
669         return new IOException(throwable);
670     }
671
672     private void waitForTermination()
673     {
674         try
675         {
676             if (!terminationLatch.await(60, TimeUnit.SECONDS))
677             {
678                 logger.warn("Timed out waiting for HTTP sender termination");
679             }
680         }
681         catch (InterruptedException e)
682         {
683             Thread.currentThread().interrupt();
684         }
685     }
686
687     private static ThreadFactory namedThreadFactory(String name)
688     {
689         return runnable -> {
690             Thread thread = new Thread(runnable, name);
691             thread.setDaemon(true);
692             return thread;
693         };
694     }
695
696     private static void sleepQuietly(long millis)
697     {
698         try
699         {
700             Thread.sleep(millis);
701         }
702         catch (InterruptedException e)
703         {
704             Thread.currentThread().interrupt();
705         }
706     }
707 }
708
```
